

APERTRAIL · DESIGN DOCUMENT

Photo spots: design & implementation plan

Status: built, in full. All eight phases of the build order in §13 shipped. [docs/features/travel.md](#) describes what the code does today; this document is why it does it that way. Where the two disagree, the feature doc is right and this one has drifted.

What changed during the build. Three things worth knowing before reading the plan as if it were the code. The fence language settled on [apt-photo-spot](#) (§5, open question 1). The conflict check in §6.3 was implemented as the more general schedule-versus-travel-time comparison rather than a light-window one, which catches the same case and the ordinary version too, and overlapping stops count. And a bug the design did not anticipate: a sub-folder setting added after a vault was already configured fell back under the pristine default root rather than the vault's own, so a relocated Travel tree scanned the wrong place for photo spots.

[getLocalizedFolderDefaults\(\)](#) now takes the saved root.

This is the feature the tagline is about. README and [docs/index.md](#) both say APERtrail is "the first dedicated travel planner for photography" and both immediately admit that nothing in the code knows about light, lenses or golden hour. Photo spots are what closes that gap.

0. What was asked for

A photo spot is a place you go to in order to make a specific picture. The source material for the requirement is a printed photo location guide (one double page per location), which is a good specification because it was written by people who actually use it in the field. A page carries:

- a title and a short orientation paragraph,
- a trivia callout ("Wusstest du...?"),
- a **main motif** with where to stand and what lens to bring,
- a **secondary motif**, often several kilometres away from the main one,
- a technique tip tied to a specific motif (long exposure, strong ND filter),
- sample photographs showing the result at the light the page recommends,
- a logistics box: address, parking, rail, bus, opening hours, entry fee, accessibility, website.

The requirement is that this survives being turned into a note, and then that a note like it can be pulled into a trip's itinerary at the right time of day. The second half is the part a printed book cannot do and is where the plugin earns its positioning.

1. Architectural placement

Photo spots become a **ninth entity type**, [photospot](#), and the **fifth member of** [TRAVEL_PLACE_TYPES](#).

Making it a place type rather than a standalone type is the whole trick. [TravelPlace](#) already carries country, city, [geoLocation](#), [address](#), [website](#), [rating](#), [visited](#) / [lastVisit](#) and tags, and every consumer of that shape is written against [TRAVEL_PLACE_TYPES](#) rather than against four hardcoded branches: [read-entities.ts](#), [create-entities.ts](#)'s [createPlaceNote\(\)](#), [visit-derivation.ts](#), [travel-dashboard-sort.ts](#), the gallery's facets, [TravelStopTargetKind](#), and the health check. A photo spot inherits all of it by being added to one

as `const` array and one folder-mapping record. The photography specific fields are then additive, exactly the way `accommodationType` / `fnbType` are additive for their kinds.

The alternatives were weighed and rejected in §10.

2. Data model

2.1 Shape

A photo spot note is a place note plus four things:

ADDITION	FORM	WHY NOT A PLAIN PROPERTY
Motifs	list of maps under <code>motifs:</code>	A spot has one main and zero or more secondary motifs, each with its own coordinates, bearing, light window and technique. Flattening this into <code>mainMotif</code> / <code>secondaryMotif</code> caps the count at two and duplicates every sub-field.
Samples	list of maps under <code>samples:</code>	A sample is an image plus the light it was taken in plus its exposure. The image alone is the existing cosmetic <code>image:</code> property, which stays what it is.
Transit	list of maps under <code>transit:</code>	The book prints one row per mode with its own icon. A list of <code>{mode, detail}</code> gives the same row per mode from a fixed mode vocabulary, mirroring <code>transport:</code> 's <code>legModeField</code> .
Access	flat top-level properties	<code>openingHours</code> , <code>entryFee</code> , <code>accessibility</code> , <code>parking</code> . Deliberately not nested under an <code>access:</code> map: Obsidian's property editor renders top-level scalars and refuses nested maps, and every one of these is a value a user will want to edit in the sidebar rather than in the YAML.

Lists of maps are already precedent (`stops:`, `nights:`, `transport:` on a Trip), and the property editor already declines to render those, so `motifs:` / `samples:` / `transit:` cost nothing that is not already paid.

2.2 Worked example

The Neuchâtel page from the source guide, as a note:

```
---
type: photospot
image: neuchatel-chateau.jpg
country: "[[Schweiz]]"
city: "[[Neuchâtel]]"
geoLocation:
  - "46.9899"
  - "6.9293"
address: 2000 Neuchâtel
website: https://www.neuchatelville.ch
rating: 5
visited: true
lastVisit: 2025-06-14
timezone: Europe/Zurich
openingHours: 24h
```

```
entryFee: none
accessibility: partial
parking: Parkhaus du Seyon, 2000 Neuchâtel
transit:
  - mode: rail
    detail: Keine direkte Bahnanbindung vorhanden
  - mode: bus
    detail: Linie 380, Haltestelle Neuchâtel Écuse, von dort ca. 5 Minuten zu Fuss
motifs:
  - name: Schloss Neuchâtel
    role: main
    geoLocation:
      - "46.9895"
      - "6.9243"
    direction: 215
    light:
      - golden-hour-evening
      - blue-hour-evening
    lens: 70-200
    gear:
      - tripod
    captured: true
    capturedOn: 2025-06-14
    note: >-
      Freie Sicht auf das Schloss hat man vom Freizeitzentrum auf dem Chemin
      de la Boine. Ein Teleobjektiv hilft dabei, den passenden Bildausschnitt
      zu finden.
  - name: Pavillon des Bains, Chez-le-Bart
    role: secondary
    geoLocation:
      - "46.9161"
      - "6.8419"
    direction: 65
    light:
      - blue-hour-morning
      - sunrise
    lens: 16-35
    gear:
      - tripod
      - nd1000
    captured: false
    technique: >-
      Um die Wasseroberfläche komplett bewegungsfrei und glatt zu
      fotografieren, benötigt man eine möglichst lange Verschlusszeit. In
      Abhängigkeit von Blende und verfügbarem Licht hilft ein starker
      ND-Filter (64x oder 1000x).
    note: >-
      Am Ende eines kleinen Steges am Westufer des Neuenburger Sees, etwa 16
```

```

    Kilometer südöstlich von Neuchâtel. Parkplatz an der Rue de Port in
    direkter Nähe zum Steg. Am Morgen fotografiert man in Richtung
    aufgehender Sonne.
samples:
  - image: neuchatel-pavillon-blue.jpg
    motif: Pavillon des Bains, Chez-le-Bart
    light: blue-hour-morning
    exposure: 30s, f/11, ISO 100, ND1000
tags:
  - Travel/PhotoSpot
---

# Neuchâtel (Neuenburg)

Neuchâtel zählt rund 45.000 Einwohner und liegt etwa 50 Kilometer westlich
von Bern am Neuenburger See.

> [!info] Wusstest du...?
> Obwohl der Kanton Neuenburg in der französischsprachigen Westschweiz
> liegt, ähnelt dessen Fahne stark der italienischen Trikolore.

```apt-photo-spot

```

## Review

Two things about that example are load bearing. The secondary motif sits 16 km from the note's own `geoLocation`, which is why a motif carries coordinates of its own rather than inheriting the note's. And the two motifs want opposite ends of the day, which is exactly the kind of thing the itinerary should be able to tell you before you drive there.

### 2.3 Field reference: note level

Property	Setting	Type	Notes
`type`	`typePropertyName`	`photospot`	Fixed value, lowercase, unquoted, one word, matching `fnb`'s precedent.
`country`	`countryProperty`	wikilink	Shared place shape.
`city`	`cityProperty`	wikilink	Shared place shape. Optional: a spot in open country has a Country and no City.
`geoLocation`	`geoLocationProperty`	`[lat, lon]` strings	The spot as a whole, used for sun times and as the map anchor. Individual motifs may override.
`address` / `website`	`addressProperty` / `websiteProperty`	text	Shared place shape, already read-only today.

```
| `rating` | `ratingProperty` | 1-5 | Shared place shape. |
| `visited` / `lastVisit` | `visitedProperty` / `lastVisitProperty` | bool / date | Shared place
shape, still derivable from finished trips. |
| `timezone` | `timezoneProperty` | IANA zone | New. See §4.3. Optional; falls back to the
device zone. |
| `openingHours` | `openingHoursProperty` | text | Free text. `24h` renders as the round-the-
clock badge. |
| `entryFee` | `entryFeeProperty` | text | Free text. Empty or `none` renders as "no entry fee".
|
| `accessibility` | `accessibilityProperty` | fixed vocabulary | `full` / `partial` / `none` /
`unknown`. |
| `parking` | `parkingProperty` | text | Free text, one line, as printed. |
| `transit` | `transitProperty` | list of maps | Sub-keys via `transitModeField` /
`transitDetailField`. |
| `motifs` | `motifsProperty` | list of maps | §2.4. |
| `samples` | `samplesProperty` | list of maps | §2.5. |
```

`image`, `icon`, `color`, `summary`, `created` and `modified` stay what they are today: cosmetic fields the templates emit and the plugin does not write.

### ### 2.4 Field reference: a motif entry

```
| Sub-key | Setting | Type | Notes |
|---|---|---|---|
| `name` | `motifNameField` | text | Required. Everything else keys off it, including a sample's
`motif` back-reference. |
| `role` | `motifRoleField` | `main` / `secondary` | Fixed vocabulary. Exactly zero or one
`main` per note; a second one is a health-check warning, not an error. |
| `geoLocation` | `motifGeoField` | `[lat, lon]` strings | Optional. Absent means "the note's
own coordinates". |
| `direction` | `motifDirectionField` | 0-359 | The bearing you shoot toward. This is the
field that makes sun position mean something. Written as degrees; a compass point (`SW`, `ONO`)
is accepted on read for hand-written notes and normalized to degrees the next time the entry is
saved. |
| `light` | `motifLightField` | list, fixed vocabulary | §2.6. Ordered best-first. |
| `season` | `motifSeasonField` | list of months | `1`-`12`, or month names. Optional. |
| `lens` | `motifLensField` | text | As printed: `70-200`, `16-35`, `Tele`. Never parsed. |
| `gear` | `motifGearField` | list, open vocabulary | `tripod`, `nd1000`, `polarizer`, ...
Rendered as chips, matched case-insensitively against a known-icon table with a neutral
fallback. |
| `technique` | `motifTechniqueField` | text | The camera-tip box on the printed page. |
| `note` | `motifNoteField` | text | Where to stand, how to get there, what the view is. |
| `captured` | `motifCapturedField` | bool | You were there and you got the shot. Not the
same claim as the note's `visited`. |
| `capturedOn` | `motifCapturedOnField` | date | Optional, only meaningful when `captured` is
true. |
```

`captured` is the field worth arguing for. A place note's `visited` answers "have I been here". For a photo spot the interesting question is "do I still owe myself this picture", and the two come apart constantly: you drive to the Pavillon des Bains, it rains, you have visited it and captured nothing. Keeping them separate is what lets the gallery say \*2 of 3 motifs captured\* and lets a future trip planner offer "spots I have been to but not shot".

### ### 2.5 Field reference: a sample entry

Sub-key	Setting	Type	Notes
`image`	`sampleImageField`	vault path or wikilink	Resolved through the existing `ui/components/image-resolve.ts`.
`motif`	`sampleMotifField`	text	Matches a motif's `name`. Unmatched values render under the spot rather than being dropped, the same way an unresolved stop link stays visible.
`light`	`sampleLightField`	fixed vocabulary	Which light this frame was made in.
`exposure`	`sampleExposureField`	text	Printed verbatim: `30s, f/11, ISO 100, ND1000`. Never parsed.
`credit`	`sampleCreditField`	text	Optional.

### ### 2.6 Fixed vocabularies

Property **names** are settings; these **values** are not, for the same reason `TRAVEL\_STATUS\_VALUES` is not: the sun-time calculation, the light badges and the itinerary's warnings all key off these exact strings.

**Light windows**, in day order:

`blue-hour-morning`, `sunrise`, `golden-hour-morning`, `day`, `overcast`,  
`golden-hour-evening`, `sunset`, `blue-hour-evening`, `night`

`overcast` is in the list because it is a real answer for some subjects (waterfalls, forests) and because "any time, as long as the sky is flat" is information the itinerary should not throw away. It is the one member with no clock window attached.

**Motif roles:** `main`, `secondary`.

**Accessibility:** `full`, `partial`, `none`, `unknown`.

**Transit modes:** `rail`, `bus`, `tram`, `boat`, `cablecar`, `foot`,  
`car`. Each maps to one Lucide icon.

## ## 3. Settings

New folder setting, following the existing convention exactly:

Setting	Default (EN)	Default (DE)
---	---	---
`photoSpotsFolder`	`4 Resources/Travel/Photo Spots`	`4 Ressourcen/Reisen/Fotospots`

Derived from the resolved `rootFolder` in `getLocalizedFolderDefaults()` like its eight siblings, independent once saved.

New property-name settings, all defaulting to their own bare name:

```
`timezoneProperty`, `openingHoursProperty`, `entryFeeProperty`,
`accessibilityProperty`, `parkingProperty`, `transitProperty`,
`motifsProperty`, `samplesProperty`.
```

New sub-key field settings, following the ``<list><Key>Field`` convention established by `stopPlaceField` and friends: the twelve `motif\*Field` keys from §2.4, the five `sample\*Field` keys from §2.5, and `transitModeField` / `transitDetailField`.

Like the trip-structure sub-key settings, the `\*Field` settings are real settings that get **no row** on the settings tab. Nineteen visible property overrides is already the practical ceiling for that tab; adding nineteen more sub-key rows would make the folders section unreachable. They stay editable in `data.json` for a vault that needs them, which is the same bargain `stopPlaceField` already made.

New behaviour setting:

Setting	Default	What it does
---	---	---
`sunTimesEnabled`	`true`	Master switch for everything in §4. Off means photo spots still work as place notes, with no sun badges, no golden-hour prefill and no light warnings.

Deliberately **not** a setting: the golden-hour and blue-hour elevation thresholds. They are a fixed convention (§4.1), and a vault that disagrees by half a degree gains nothing it can use.

## ## 4. Sun and light

This is the part a printed guide cannot do, and it needs no network and no API key. Sun position is arithmetic.

### ### 4.1 `src/models/solar.ts`

A single dependency-free module, no Obsidian imports, therefore trivially unit-testable:

```
sunTimes(date, lat, lon) -> { blueHourMorningStart, sunriseStart, sunriseEnd, goldenHourMorningEnd, solarNoon,
goldenHourEveningStart, sunsetStart, sunsetEnd, blueHourEveningEnd }
```

```
sunPosition(instant, lat, lon) -> { altitude, azimuth }
```

```
lightRelation(azimuth, motifDirection) -> 'back' | 'side' | 'front' | 'unknown'
```

Standard NOAA solar-position algorithm. Boundaries follow the convention photographers actually use:

```
| Window | Sun elevation |
|---|---|
| Blue hour | -6° to -4° |
| Sunrise / sunset | -0.833° (refraction-corrected horizon) |
| Golden hour | -4° to +6° |
| Day | above +6° |
| Night | below -6° |
```

``lightRelation()`` compares the sun's azimuth against the bearing you shoot toward: within 45° of the shooting direction means the sun is behind the subject (backlit), within 45° of the opposite bearing means it is behind you (front lit), otherwise side lit. Three buckets, because a fourth would imply a precision the input does not have.

Two honest limitations to state in the UI rather than hide: this is geometry, not weather, and it assumes a flat horizon. A spot in a valley loses its golden hour to the ridge line, and no amount of arithmetic knows that. The note's ``note`` field is where a human writes "the sun clears the ridge about 40 minutes after sunrise".

### ### 4.2 What consumes it

- The photo-spot block renders today's windows for the spot, and for a motif with a ``direction``, whether its preferred light is front, side or back lit on that bearing.
- The itinerary prefills a stop's ``from`/`to`` when the stop is a photo spot (§6.2).
- The itinerary's day rows get a sun band (§6.1).

### ### 4.3 Timezones

Sun times are computed in UTC and rendered in the spot's zone. The spot's zone is ``timezone:`` when the note carries it, otherwise the device's zone. A vault used entirely at home never notices; a vault with spots in Iceland and Japan needs it, and getting this wrong is the kind of bug that shows up as "the plugin says golden hour is at 03:40" and nothing else.



Anything written into a note with a clock time in it is written **quoted**, per the datetime rule in ``docs/design/data-model.md``. An unquoted ``2026-06-14T05:12`` becomes a native ``Date`` in Obsidian's YAML parser and loses its time on the way back out.

## 5. The ``apt-photo-spot`` code block

An empty fence in a photo-spot note, taking no arguments, working out what to render from ``ctx.sourcePath``, exactly like the two blocks that already exist:

```

````
```apt-photo-spot

```

```

````

```

It renders, in order:

1. **Motif cards**, main first, then secondary in note order. Each card carries the motif name, a role badge, the light chips in the vocabulary order of §2.6 with today's clock window resolved beside each, lens and gear chips, the `note` text, and the `technique` text in a tip box with a camera icon. A motif with its own coordinates shows the distance and bearing from the note's anchor, which is what makes "16 km southeast" visible instead of buried in prose.
2. **A capture strip** per motif: captured / not captured, with the date, and a one-click toggle that writes `captured` and `capturedOn` back.
3. **Samples**, as a thumbnail row grouped by motif, opening in the existing `ui/components/lightbox.ts`, with the exposure line under each.
4. **A logistics band**: parking, transit rows by mode, opening hours, entry fee, accessibility, website. This is the printed page's grey box, and the icon set is chosen to match what that box does, because it is already the right design.
5. **A sun panel** for a chosen date, defaulting to today, with a date stepper. Blue hour, sunrise, golden hour, solar noon, golden hour, sunset, blue hour, plus each motif's preferred windows highlighted.

Like the itinerary block it is an editing surface, not just a view: the capture toggle, "add motif", per-motif edit and reorder, and "add sample". Every mutation reads the whole note, mutates the model, and writes it back through one `updatePhotoSpotNote()`, so there is one save path and no partial writes, which is the lesson `write-trip.ts` already paid for.

On the fence language. The block is `apt-photo-spot`, not `travel-photo-spot`. The two existing blocks kept their `travel-` prefix only because the strings already live in users' notes and a rename would orphan every one of them; that argument cannot apply to a block that does not exist yet.

The argument for consistency with the old two loses to what a photo spot actually is. `travel-itinerary` and `travel-related-trips` are both about trips: one renders a trip, the other renders the trips that touched a place. Neither means anything without a trip. A photo spot does. A spot note is useful with no trip planned and no trip ever taken, and the block is where that usefulness lives: exporting one spot as a field-ready PDF, checking tomorrow's golden hour, working through the motifs you still owe yourself. Naming it `travel-` would file it under a relationship it does not have.

So the vault ends up with two prefixes, and that is the honest state of things: `travel-` marks the blocks that were carried over, `apt-` marks the blocks this plugin wrote. Every block added from here on takes `apt-`.

6. Trip integration

A photo spot is a place type, so `TravelStopTargetKind` gains it and a spot can already be an itinerary stop the moment §1 lands. The rest of this section is what makes that stop worth having.

6.1 The sun band

Each day row in `travel-itinerary` gets a thin horizontal band behind the time gutter showing that day's light at the day's location: night, blue, golden, day, golden, blue, night. The day's location is the first stop of the day that has coordinates. A day with no located stop gets no band rather than a wrong one.

6.2 Golden-hour prefill

When you add a stop and pick a photo spot, the stop dialog's `from` / `to` prefill from the spot's main motif's first `light` value on that date, rather than from the day's date at midnight. Picking the Pavillon des Bains for 14 June prefills 04:52 to 05:31, not 00:00.

The prefill is a suggestion, always overwritable, and it never rewrites a time you already set. Nothing writes derived values back into a note as a side effect of editing another note; that rule holds here.

6.3 Light conflicts

Two stops on the same day that both want `golden-hour-evening`, at spots 40 minutes apart, is a plan that cannot happen. The itinerary flags it inline: a warning chip on both rows, naming the other spot. It does not reorder anything, does not refuse the entry, and does not nag. The plugin's job here is to make the conflict visible at planning time instead of at 19:40 in a car.

The rule. Two stops conflict when their light windows overlap and the straight-line distance between them, walked at 4 km/h, takes longer than the gap between the two stops. Walking speed, deliberately: it is the one assumption that holds everywhere. A rule tuned to driving is wrong on an island with no rental car, wrong in a city centre, and wrong on a ridge line with no road at all, and being wrong there means staying silent about a plan that cannot happen. Walking over-warns instead, which is the survivable direction for a chip you can ignore.

Straight-line distance from the two `geoLocation` pairs, and the warning says both parts out loud: how far apart, and that it is a line rather than a route. A road-network estimate would need a routing service and a network call, and this plugin does neither.

6.4 Shot list

A trip's "what do I still owe myself here" view: every photo spot the trip stops at, every motif at those spots, and each motif's `captured` state. Rendered at the bottom of the itinerary block. It is a read of existing data, not a new property on the trip, which is why it can ship in the same phase as §6.1 rather than needing its own frontmatter.

7. Gallery and dashboard

Gallery. A ninth type-filter chip. The card shows the spot's image, its star rating, and a meta row of: city or country, motif count with captured count, and the main motif's first light window as a chip. Photo-spot-only facets, appearing when the type filter is Photo Spot and cleared when it moves away, matching how the Trip-only facets already behave:

- light window,
- season,
- captured state (all / fully captured / partly / none),

- accessibility,
- has samples.

Each is built from values actually present in scope and is not rendered when it has nothing to offer, which is the existing facet rule.

Dashboard. An eighth section, "Photo spots", placed directly under Trips rather than at the bottom: on a product whose positioning is photography, this is the section people open the dashboard for. It sorts by rating desc, falling back to most-recently-visited, like the other place types.

The stats row grows to a fifth tile: photo spots captured / total, counting a spot as captured when its main motif is. It sits next to the landmarks tile it is shaped like.

Quick actions gain "New photo spot" in second position, right after "New trip". The existing ordering rule is "how often you reach for one", and on this product that is where it lands.

8. Creation, commands, health check

`NewPlaceModal` already takes a `kind` and parameterizes title, icon and create-function by it, so a photo spot needs three record entries and an icon (`camera`), not a new modal file. The modal collects title, Country and City, and writes minimal frontmatter, the same as the other four place types. Motifs are added afterwards in the block, where there is room for them.

`createPhotoSpotNote()` joins the four existing wrappers over `createPlaceNote()`. The new note's body carries the `apt-photo-spot` block and the `travel-related-trips` block.

One new command, "New photo spot", taking the count to twelve.

The health check widens from eight folders to nine. Two photo-spot-specific checks join it, both warnings rather than errors, because a half-filled spot is a normal state and not a broken note:

- more than one motif with `role: main`,
- a sample whose `motif` matches no motif on the note.

9. Internationalization

Every new string goes in **both** `en.ts` and `de.ts` or `tests/translation-keys.test.ts` fails. New key groups:

- `photoSpot.*` for the block: section headings, motif roles, capture toggle, sun panel labels, distance and bearing formatting.
- `photoSpot.light.*` for the nine light windows.
- `photoSpot.transit.*` for the seven transit modes.
- `photoSpot.accessibility.*` for the four accessibility values.
- `photoSpot.gear.*` for the known gear chips, with the raw value shown verbatim when there is no translation, so an unknown chip degrades to what the note says rather than to a missing-key string.
- `modals.newPhotoSpotModal.*`, `commands.newPhotoSpot`, `gallery.type.photospot`, `dashboard.sections.photoSpots`, `dashboard.stats.photoSpotsCaptured`, `settings.folders.photoSpots`.

The light-window vocabulary is a place to be careful: the values in the note stay English identifiers (`golden-hour-evening`), and only their labels are translated. A German vault that wrote `goldene-stunde-abends` into its notes would be unreadable by the same vault switched to English, which is exactly the trap `TRAVEL_STATUS_VALUES` avoids by being fixed.

10. Options considered

A. Extend Landmark and Location with optional photo fields. Any place note carries `photoSpot: true` plus the extra fields; a gallery facet surfaces them. *Rejected:* it makes "is this a photo spot" a property rather than an identity, which breaks the codebase's one hard identification rule (folder **and** type, always). It also puts twenty photography fields on the shared `TravelPlace` shape that four of five kinds never use, and gives the dashboard no honest way to count them.

B. Photo spot as a sub-entity of a parent place. A spot note points at a parent Landmark or City, so "Neuchâtel" owns two spots. *Rejected:* it adds a second containment hierarchy alongside Country/State/City for no gain. The printed page's structure is already captured by the `motifs:` list, which is the same idea without a second level of notes to keep in sync. It would also mean a photo spot could not be an itinerary stop without deciding whether the stop points at the parent or the child.

C. A ninth entity type inside `TRAVEL_PLACE_TYPES` . Recommended and designed above. It costs one array entry, one folder-mapping entry, one folder setting and one modal record to get the entire existing place machinery, and it keeps the photography fields where only photo spots pay for them.

The precedent points the same way. City and State were added after the first draft to give Country a real hierarchy, and the single generic "Ort" was split into four place types. Both times the answer was a new type inside the existing shape, not a flag on an old one.

11. Open questions

One is open. The other five were settled before implementation started and are recorded below it, because the reasoning is worth more than the verdict.

1. **Season as months or as named seasons?** Months are unambiguous across hemispheres and sort; named seasons read better and are what a photographer actually says. The design writes months and renders them as a compact range, which is a rendering decision that can be revisited without touching a note. Deliberately left open: nothing in phases 0 to 5 depends on it, and the answer will be obvious after a few dozen real spots exist.

Resolved before the build

~~2. Fence language: `travel-photo-spot` or `apt-photo-spot` ?~~ **Resolved:** `apt-photo-spot` . The two old blocks are both about trips and mean nothing without one. A photo spot is useful with no trip planned and none ever taken, and the block is where that usefulness lives, down to exporting a single spot as a field-ready PDF. Naming it `travel-` would file it under a relationship it does not have. See §5. Consequence: every block added from here on takes `apt-` , and the vault carries two prefixes that mean two different things.

~~3. Does `visited` stay derived for photo spots, and should it touch `captured` ?~~ **Resolved:** `visited` **never touches** `captured` . Being there is not getting the shot. The plugin must never claim a picture exists; only the person who took it may set that field.

~~4. `direction` as degrees or compass points?~~ **Resolved:** **degrees.** Compass points are accepted on read so a hand-written note is not punished, and normalized to degrees on the next save.

~~5. Should a motif be linkable?~~ **Resolved: no, not now.** A motif stays a YAML entry, so nothing can wikilink to "the Pavillon at blue hour". Splitting motifs into notes later is a migration, not a redesign, which is what makes deferring it cheap.

~~6. How far apart is "a conflict" in §6.3?~~ **Resolved: straight-line distance at 4 km/h walking, compared against the gap between the two stops.** Walking is the one speed assumption that holds on an island with no rental car, in a city centre and on a road-free ridge alike. It over-warns rather than staying silent, which is the survivable direction for an advisory chip. See §6.3.

12. Deliberately out of scope

- **Weather.** No forecast, no cloud cover, no API. The sun math is offline and stays offline.
- **A map view.** Photo spots make the absence of one more obvious, and they do not fix it. `geoLocation` is still stored and still not drawn. Distance and bearing between two coordinate pairs is arithmetic and is in scope; rendering a map is not.
- **Terrain-aware horizons.** See §4.1.
- **Moon phase and Milky Way position.** The same arithmetic could do both, and astro is a real photography use case, but it is a separate feature with its own vocabulary. Not in this design.
- **Automatic EXIF import** from the samples' image files.
- **Writing `captured` back from a finished trip.** See open question 2.

13. Build order

Each phase is independently shippable and leaves the plugin in a working state.

| PHASE | CONTENTS | WHY HERE |
|-------|--|--|
| 0 | <code>photospot</code> in <code>TRAVEL_ENTITY_TYPES</code> and <code>TRAVEL_PLACE_TYPES</code> , <code>photoSpotsFolder</code> setting plus defaults, validation and settings-tab row, <code>TRAVEL_PLACE_FOLDER_SETTING</code> entry, <code>createPhotoSpotNote()</code> , <code>NewPlaceModal</code> records, the command, health check widened to nine folders, en/de keys. | Photo spots exist as place notes and appear in the gallery and dashboard with zero new UI code. Everything after this is additive. |
| 1 | The new property and sub-key settings, <code>readPhotoSpot()</code> extensions in <code>read-entities.ts</code> , <code>TravelPhotoSpot</code> fields on the model, <code>updatePhotoSpotNote()</code> writer, unit tests over parsing and writing. | The data model is real and tested before anything renders it. |
| 2 | <code>apt-photo-spot</code> block: motif cards, samples with lightbox, logistics band. Read-only. | The printed page now has a digital equivalent. Shippable on its own and the first thing a user can see. |
| 3 | <code>models/solar.ts</code> plus its tests, the sun panel in the block, light chips resolved to clock times, <code>lightRelation()</code> badges. | Pure arithmetic with no UI dependency, tested independently, then wired into one surface. |
| 4 | The block becomes an editing surface: add/edit/reorder motifs, add samples, capture toggle. | Editing lands after the shape has been proven by reading real notes. |
| 5 | Itinerary integration: sun band, golden-hour prefill, shot list. | Needs both the solar module and photo spots as stop targets. |

| PHASE | CONTENTS | WHY HERE |
|-------|---|--|
| 6 | Gallery photo-spot facets, dashboard section ordering and the fifth stats tile. | Polish over a feature that already works. |
| 7 | Light-conflict warnings. | The rule is settled (§6.3). It lands last because it is the only piece that can be wrong in a way that annoys rather than misleads, and it wants real trips to be tuned against. |

14. Documentation to update

| FILE | WHAT |
|---|---|
| <code>docs/features/travel.md</code> | Entities table (nine types), Folders table, frontmatter table, dashboard section (eight sections, five tiles, quick-action order), gallery (type filter, facets), health check (nine folders, three occurrences), commands table (twelve), settings counts. Remove "No photography features yet" from "What APERtrail does not have". |
| <code>docs/design/data-model.md</code> | The nine fixed type values, the place-shape section, the derived-fields table, the code-block-languages section. |
| <code>docs/design/settings-reference.md</code> | New folder row, new property rows and count, the new sub-key settings in "not on any tab", the new fixed vocabularies in "deliberately not settings". |
| <code>docs/design/architecture.md</code> | Entry-point tree (twelve commands, three code-block processors), code-block table, i18n note. |
| <code>docs/index.md</code> | Opening paragraph, "where to start" table row for this document, the health-check row. |
| <code>docs/usage.md</code> | Quick actions, "add places", the eight/nine folder counts in §7 to §9. |
| <code>docs/templates/index.md</code> and <code>docs/templates/Template - Photo Spot.md</code> | New template, table row, "each of the nine entity types". |
| <code>README.md</code> | The tagline stops being aspirational. |
| <code>CHANGELOG.md</code> | One entry per shipped phase. |
| <code>CLAUDE.md</code> | The code-block-languages bullet. It currently reads as "the fence languages are still <code>travel-*</code> "; it becomes "the two carried-over blocks keep <code>travel-</code> , every new block takes <code>apt-</code> ". |
| <code>docs/design/travel-module-plan.md</code> , <code>trip-model-redesign.md</code> | Historical. Do not retrofit. At most a pointer line, the way §3's Trip row already points at the redesign. |

15. Test coverage to add

- `solar.test.ts`: known sunrise/sunset pairs for a handful of latitudes and dates against published values, the polar edge cases (no sunrise, no sunset), and `LightRelation()`'s three buckets across the wrap at 0°.
- `photo-spot-parse.test.ts`: motif and sample lists, missing sub-keys, a motif with no coordinates falling back to the note's, an unmatched sample `motif`, a second `role: main`.

- `photo-spot-write.test.ts`: round-tripping a note without touching its body, quoted datetimes, omitted-not-empty optional fields.
- `translation-keys.test.ts` covers the new keys by construction.
- The health-check test's folder scan goes from eight to nine.